DEshort

Differential expression analysis aims at distinguish the cluster of genes that are over- or under- represented in each sample. Comparing case and control samples aims at identifying which are the genes correlated to the disease.

0. importing data

Import the dataframe and filter unuseful information

```
data <- read.table("R_steps/reads_gene_counts.txt",
                   header = TRUE,
                   sep = "\t")
colnames(data)
```

```
##  [1] "Geneid"     "Chr"        "Start"      "End"        "Strand"
##  [6] "Length"     "SRR7821918" "SRR7821919" "SRR7821920" "SRR7821921"
## [11] "SRR7821922" "SRR7821923" "SRR7821924" "SRR7821925" "SRR7821927"
## [16] "SRR7821937" "SRR7821938" "SRR7821939" "SRR7821940" "SRR7821941"
## [21] "SRR7821942"
```

```
dim(data)
```

```
## [1] 78334    21
```

results in a table with 78334 obs for 21 variables (15 of which are our samples). Then, filter for the columns corresponding to the samples and ignore the rest of the information. Specifically, remove the columns containing Chr, Start, End, Strand and Length.

```
data <- data[,-2:-6]
colnames(data)
```

```
##  [1] "Geneid"     "SRR7821918" "SRR7821919" "SRR7821920" "SRR7821921"
##  [6] "SRR7821922" "SRR7821923" "SRR7821924" "SRR7821925" "SRR7821927"
## [11] "SRR7821937" "SRR7821938" "SRR7821939" "SRR7821940" "SRR7821941"
## [16] "SRR7821942"
```

Create a matrix ready to feed into DESeq2. It has to be "numeric".

```
rownames(data) <- data[,1]   # set first column as row names
data <- data[ , -1]          # remove the first column
```

5. Exploratory data analysis (DESeq2)

Install DESeq2

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager")

BiocManager::install("DESeq2")
```

Get the sample information into a table containing the sample name and to which group it belongs.

```
sample_info <- read.delim(
  "R_steps/sample_info.txt",
  header = TRUE,
  sep = " ",
  stringsAsFactors = FALSE,
  check.names = FALSE
)
```

Now give the data to DEseq2 so that it can create a dds (DESeq Data Set that is created from the data matrix).

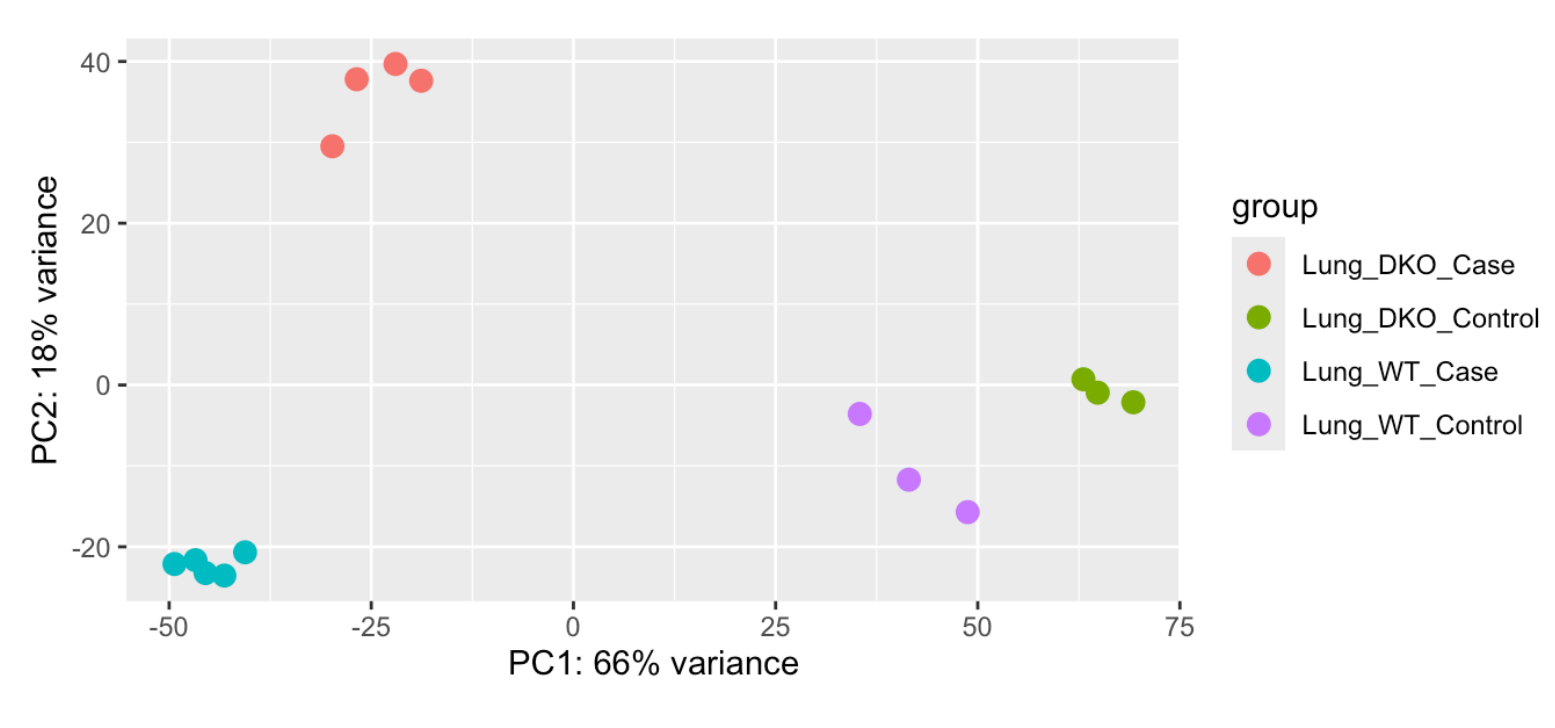
```
dds <- DESeq2::DESeqDataSetFromMatrix(countData = data,
                              colData = sample_info,
                              design= ~ Group)  # UNUSEFUL do not change df to matrix and then df again
```

Now remove the dependence of the variance on the mean by running `DESeq2::vst()` on dds. For visualization, we typically do not want to consider the experimental groups so set `blind = TRUE`.

plot the PCA to check whether it is all right with the data and it is possible to distinguish in the 4 clusters where they belong to.

```
# PCA using top 500 features by variance
DESeq2::plotPCA(vsd, intgroup = "Group")
```

```
## using ntop=500 top features by variance
```



6. Differential Expression analysis

```
dss <- DESeq2::DESeq(dds)
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
dss
```

```
## class: DESeqDataSet
## dim: 78334 15
## metadata(1): version
## assays(4): counts mu H cooks
## rownames(78334): ENSMUSG00000104478 ENSMUSG00000104385 ...
##   ENSMUSG00002074899 ENSMUSG00002076890
## rowData names(30): baseMean baseVar ... deviance maxCooks
## colnames(15): SRR7821918 SRR7821919 ... SRR7821941 SRR7821942
## colData names(3): Sample Group sizeFactor
```

```
DESeq2::resultsNames(dss)
```

```
## [1] "Intercept"
## [2] "Group_Lung_DKO_Control_vs_Lung_DKO_Case"
## [3] "Group_Lung_WT_Case_vs_Lung_DKO_Case"
## [4] "Group_Lung_WT_Control_vs_Lung_DKO_Case"
```

check resultsNames and extract the comparisons of interest

To count how many are differentially expressed, check how many are not NA in the padj column and lower than the classic 0.05 threshold.

Check the number of differentially expressed genes in each comparison with *padj < 0.05*

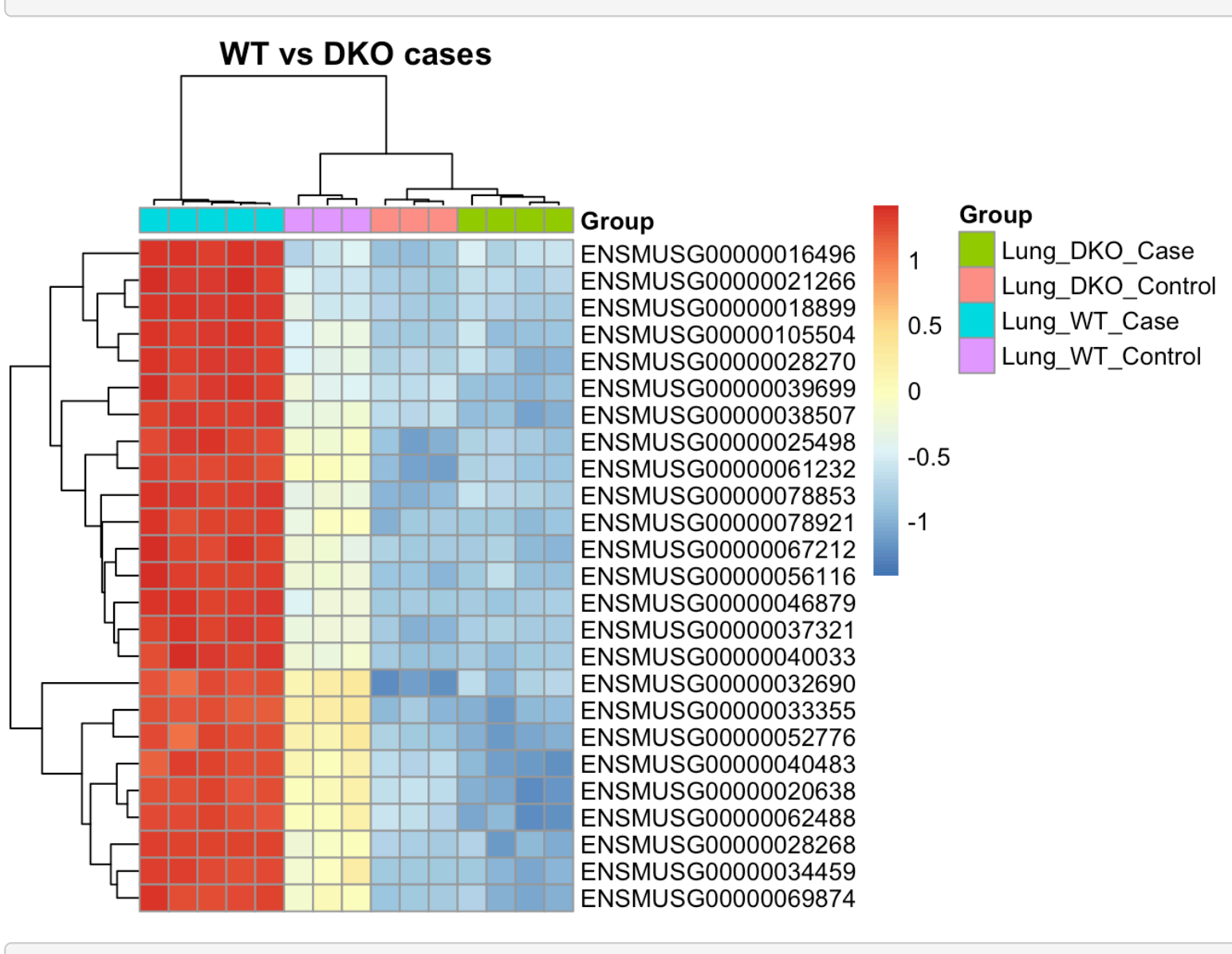
- **WT case and DKO case**

```
get_results <- function(DEout, this, to_that){
  res_comp <- DESeq2::results(DEout, contrast = c("Group", this, to_that)) # extract comparison

  filt_comp <- res_comp[res_comp$padj < 0.05 & ! is.na(res_comp$padj), ] # filter for padj

  return(filt_comp)
}

WT_Case_DKO_Case <- get_results(dss, "Lung_WT_Case", "Lung_DKO_Case")
WT_Control_WT_Case <- get_results(dss, "Lung_WT_Control", "Lung_WT_Case") # WT control vs WT case
DKO_Control_DKO_Case <- get_results(dss, "Lung_DKO_Control", "Lung_DKO_Case")
```

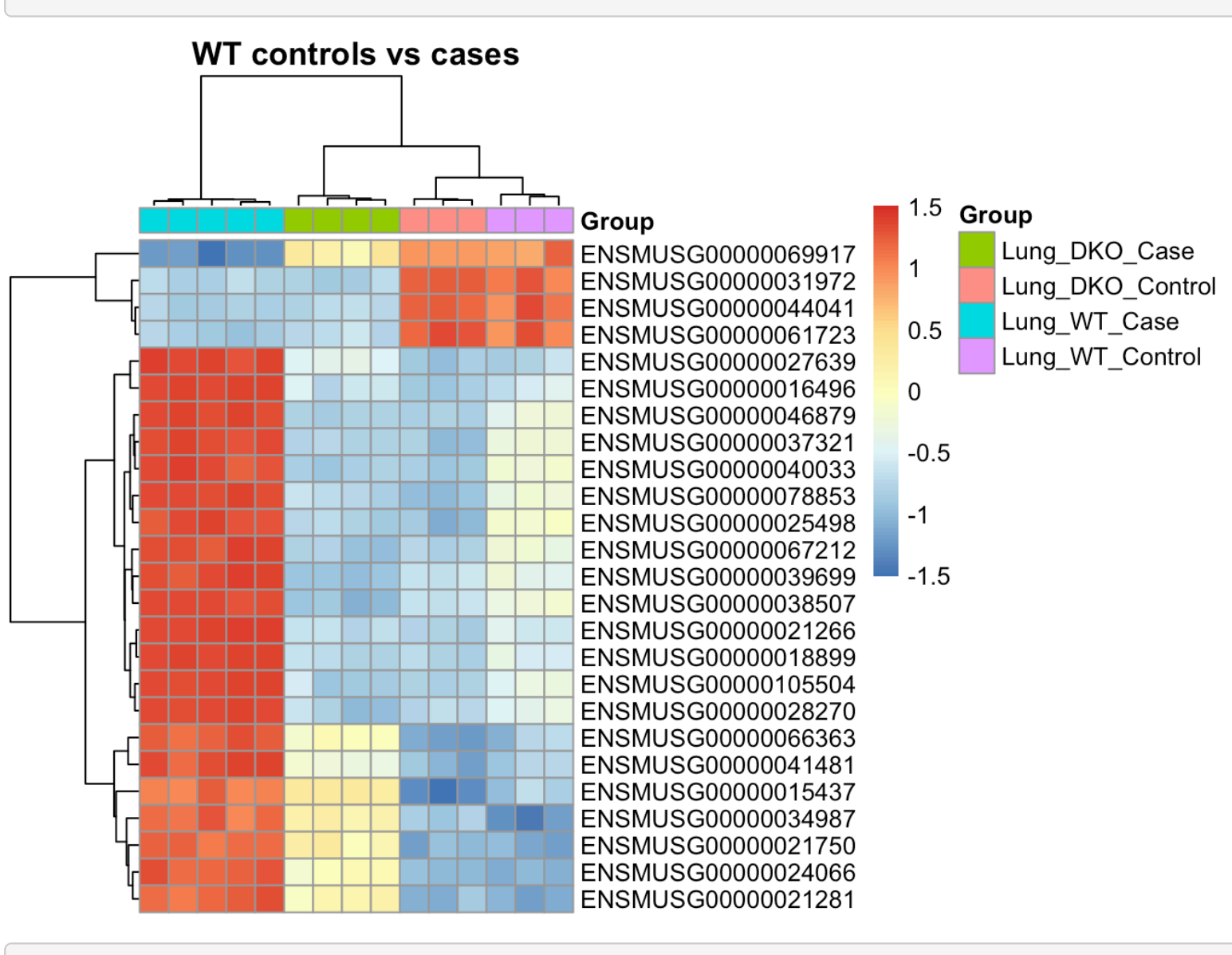
Heatmap

Produce heatmaps to visualize the differences in the expression between groups.

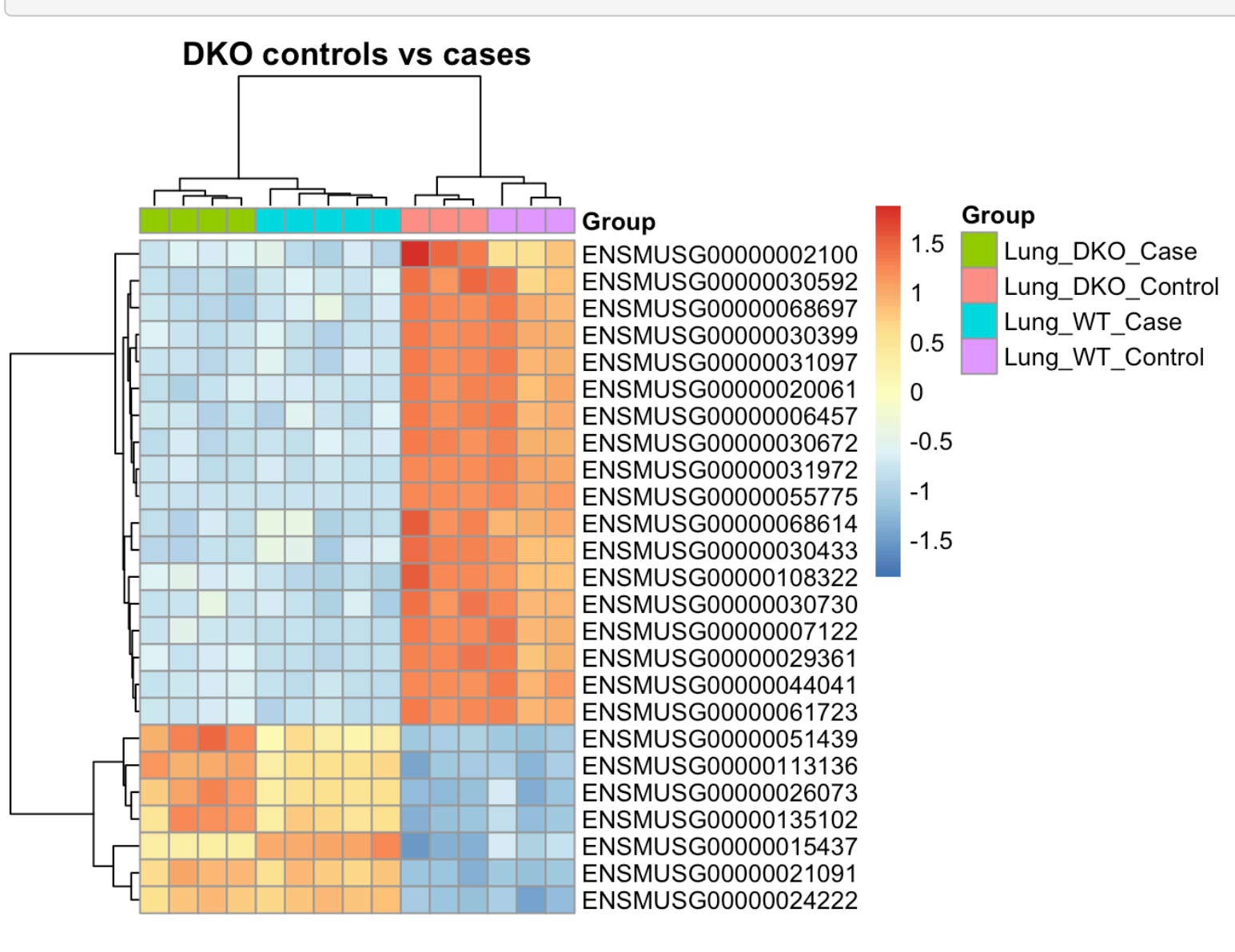
```
# 1. Extract VSD matrix
vsd_mat <- SummarizedExperiment::assay(vsd)

heatplot <- function(comparison, samples, vsd, title, leg = TRUE){
  # 2. Select significant genes for the comparison
  filtered <- comparison[(comparison$log2FoldChange > 1 | comparison$log2FoldChange < -1), ]
  filtered <- filtered[order(filtered$padj),]
  filtered <- filtered[1:25,]
  heat_matrix <- vsd[rownames(filtered),]

  # 3. Build annotation for the comparison
  ann <- samples[, "Group", drop = FALSE]
  rownames(ann) <- samples$Sample
  ann <- ann[colnames(heat_matrix), , drop = FALSE]

  # 4. Plot heatmap
  library(pheatmap)
  pheatmap(heat_matrix,
           annotation_col = ann,
           show_rownames = TRUE,
           fontsize_col = 10,
           clustering_distance_rows = "euclidean",
           clustering_distance_cols = "euclidean",
           clustering_method = "complete",
           scale = "row",
           main = title,
           annotation_legend = leg,
           show_colnames = FALSE)
}

heatplot(WT_Case_DKO_Case, sample_info, vsd_mat, title = "WT vs DKO cases")
```



```
heatplot(WT_Control_WT_Case, sample_info, vsd_mat, title = "WT controls vs cases")
```



```
heatplot(DKO_Control_DKO_Case, sample_info, vsd_mat, title = "DKO controls vs cases")
```



SUMMARY REPORT TABLE 1

How many genes are differentially expressed (DE) in the pairwise comparisons you selected (e.g. padj < 0.05)?

```
paste0("There has been found ", nrow(WT_Case_DKO_Case), " DE genes in WT vs DKO case comparison having padj < 0.0
5.")
```

```
## [1] "There has been found 7976 DE genes in WT vs DKO case comparison having padj < 0.05."
```

```
paste0("There has been found ", nrow(WT_Control_WT_Case), " DE genes in WT control vs case comparison having padj
< 0.05.")
```

```
## [1] "There has been found 11055 DE genes in WT control vs case comparison having padj < 0.05."
```

```
paste0("There has been found ", nrow(DKO_Control_DKO_Case), " DE genes in DKO control vs case comparison having p
adj < 0.05.")
```

```
## [1] "There has been found 11405 DE genes in DKO control vs case comparison having padj < 0.05."
```

How many of the DE genes are up-regulated vs down-regulated meaningfully (padj < 0.05)?

```
paste0("For WT vs DKO case are ", sum(WT_Case_DKO_Case$log2FoldChange > 1), " upregulated and ", length(WT_Case_D
KO_Case$log2FoldChange) - sum(WT_Case_DKO_Case$log2FoldChange > -1), " downregulated.")
```

```
## [1] "For WT vs DKO case are 1909 upregulated and 2063 downregulated."
```

```
paste0("For WT controls vs cases are ", sum(WT_Control_WT_Case$log2FoldChange > 1), " upregulated and ", length(W
T_Control_WT_Case$log2FoldChange) - sum(WT_Control_WT_Case$log2FoldChange > -1), " downregulated.")
```

```
## [1] "For WT controls vs cases are 3961 upregulated and 2805 downregulated."
```

```
paste0("For DKO control vs case are ", sum(DKO_Control_DKO_Case$log2FoldChange > 1), " upregulated and ", length(
DKO_Control_DKO_Case$log2FoldChange) - sum(DKO_Control_DKO_Case$log2FoldChange > -1), " downregulated.")
```

```
## [1] "For DKO control vs case are 3874 upregulated and 2911 downregulated."
```

Based on the original publication, select 2-3 genes that are of particular interest and investigate their expression level. make **SUMMARY REPORT TABLE 2** for the selected genes with name, ENSEMBL name, pathway, count.

```
common <- rownames(WT_typeII)[which(! rownames(WT_typeII) %in% rownames(DKO_typeII))]

DKO_typeI[which(rownames(DKO_typeI) == "ENSMUSG00000052776"),]
```

After comparing the DE genes in the three different comparisons, I choose four representative ones from different BP families that explain best what has been concluded in the paper.

- Oas1a (type I) ENSMUSG00000052776 present in all three comparisons, so this gene function is related to the disease and not influenced by the DKO
- Ifit1 (type I) ENSMUSG00000034459 not present in WT control and case, but not present in DKO case, so affected by DKO
- Gbp5 (type II) ENSMUSG00000105504 ||
- Irf1 (type II) ENSMUSG00000018899 ||

Oas1a (type I) ENSMUSG00000052776 Ifit1 (type I) ENSMUSG00000034459 Gbp5 (type II) ENSMUSG00000105504 Irf1 (type II) ENSMUSG00000018899

take the vsd information on the corrected variance and plot those

```
gene_annot <- data.frame(
  Gene = c(
    "ENSMUSG00000052776",
    "ENSMUSG00000034459",
    "ENSMUSG00000105504",
    "ENSMUSG00000018899"
  ),
  Label = c(
    "Oas1a (type I)\nENSMUSG00000052776",
    "Ifit1 (type I)\nENSMUSG00000034459",
    "Gbp5 (type II)\nENSMUSG00000105504",
    "Irf1 (type II)\nENSMUSG00000018899"
  )
)
# "StatII (type I)\nENSMUSG00000040033"

library(dplyr)

df_long <- df_long %>%
  left_join(gene_annot, by = "Gene")

# convert Label to a factor with the specific order
df_long$Label <- factor(df_long$Label, levels = c(
  "Oas1a (type I)\nENSMUSG00000052776",
  "Ifit1 (type I)\nENSMUSG00000034459",
  "Gbp5 (type II)\nENSMUSG00000105504",
  "Irf1 (type II)\nENSMUSG00000018899"
))


comparisons <- list(
  c("Lung_WT_Control", "Lung_WT_Case"),
  c("Lung_DKO_Control", "Lung_DKO_Case"),
  c("Lung_WT_Case", "Lung_DKO_Case")
)

library(ggplot2)
library(ggpubr)

# These colors help distinguish between WT and DKO groups clearly [1, 2]
group_colors <- c(
  "Lung_WT_Control"  = "#CCCCCC", # Light Grey
  "Lung_WT_Case"     = "#00008B", # Dark Blue
  "Lung_DKO_Control" = "#FFD700", # Gold/Yellow
  "Lung_DKO_Case"    = "#FF4500"  # Orange-Red
)

# 2. Updated Plotting Code
ggplot(df_long, aes(x = Group, y = Expression, fill = Group)) +
  # Add transparency (alpha) to the boxplot to make it look cleaner
  geom_boxplot(outlier.shape = NA, alpha = 0.7, color = "black") +
  # Increase jitter transparency to avoid clutter while showing every replicate [5]
  geom_jitter(width = 0.15, size = 1.5, alpha = 0.5) +

  # Maintain your factor-based ordering for Oas1a, Ifit1, Gbp5, and Irf1 [6, 7]
  facet_wrap(~ Label, scales = "free_y") +

  stat_compare_means(
    comparisons = comparisons,
    method = "wilcox.test",
    label = "p.signif",
    hide.ns = TRUE,
    step.increase = 0.12 # Prevents p-value labels from overlapping
  ) +

  # Switch to theme_minimal to remove the heavy grey background [3]
  theme_minimal(base_size = 12) +
  theme(
    # Remove the grey boxes behind the facet labels (gene names)
    strip.background = element_blank(),
    strip.text = element_text(face = "bold", size = 10),

    # Clean up the axes and remove background clutter
    panel.grid.minor = element_blank(),
    panel.grid.major.x = element_blank(),
    axis.line = element_line(color = "black"),
    axis.text.x = element_text(angle = 45, hjust = 1),

    legend.position = "none" # Groups are already labeled on the X-axis
  ) +

  # Apply the custom color scale
  scale_fill_manual(values = group_colors) +

  labs(
    title = "VST Expression of Selected Interferon-Related Genes",
    y = "VST Normalized Expression",
    x = NULL
  )

ggsave(
  filename = "IFN_genes_VST_boxplot.png", # You can also use .pdf, .jpeg, or .tiff
  width = 12,                          # Width in inches
  height = 8.5,                          # Height in inches
  dpi = 300                            # High resolution for the report
)
```

genes from paper

```
# Install database
if (!require("org.Mm.eg.db", quietly = TRUE))
  BiocManager::install("org.Mm.eg.db")

library(org.Mm.eg.db)
library(AnnotationDbi)

# function to obtain ENSEMBL names from their name
gfilter <- function(symbols, dss){
  ens_ids <- mapIds(
    org.Mm.eg.db,
    keys      = symbols,
    column    = "ENSEMBL",
    keytype   = "SYMBOL",
    multiVals = "first"
  )

  dss_fil <- dss[grepl(paste(unname(ens_ids), collapse="|"), row.names(dss), ignore.case = TRUE), ]

  # add gene name to the data frame
  gene_map_one <- data.frame(
    ENSEMBL = unname(ens_ids),
    name    = names(ens_ids),
    stringsAsFactors = FALSE
  )

  dss_fil$name <- gene_map_one$name[match(rownames(dss_fil), gene_map_one$ENSEMBL)]

  return(dss_fil)
}

# list of gene names to convert into ENSEMBLE codes
## L5 (type I IFN)
symbols <- c(
  "Ifnb1", "Ifit1", "Ifit3", "Oas1a", "Oas2", "Oas3", "Oasl1", "Oasl2", "Mx1", "Stat2", "Irf7", "Irf9"
)

Case_typeI <- gfilter(symbols, WT_Case_DKO_Case)
WT_typeI <- gfilter(symbols, WT_Control_WT_Case)
DKO_typeI <- gfilter(symbols, DKO_Control_DKO_Case)


## L7 (type II IFN)
symbols <- c(
  "Ifng", "Irf1", "Irf2", "Irf8", "Il12rb1", "Il12rb2", "Tap1", "Tap2", "Gbp2", "Gbp3", "Gbp4", "Gbp5", "Gbp6", "
Gbp8", "Gbp9", "Gbp10", "Gbp11"
)

Case_typeII <- gfilter(symbols, WT_Case_DKO_Case)
WT_typeII <- gfilter(symbols, WT_Control_WT_Case)
DKO_typeII <- gfilter(symbols, DKO_Control_DKO_Case)
```

7. Overrepresentation analysis

Hypergeometric test to check overrepresentation

The databases for overrepresentation test where you can get the pathway categorization are: - GO - KEGG - Reactome - Wikipaths

I would use GO and KEGG (more curated? check why don't go randomly)

Install clusterProfiler

```
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

if (!require("clusterProfiler", quietly = TRUE))
  BiocManager::install("clusterProfiler")
# the link to enrichplot is broken in bioconductor, install directly
if (!require("enrichplot", quietly = TRUE))
  BiocManager::install("enrichplot", force = TRUE)
# without updating the other packages (might create problems with dependencies)

library("clusterProfiler")
```

Download Bioconductor package with genome wide annotation for your species

```
if (!require("org.Mm.eg.db", quietly = TRUE))
  BiocManager::install("org.Mm.eg.db")
```

Call enrichGO

```
# Select significant genes WT control and cases and run the enrichment via Gene Ontology
ego_gen <- function(comparison, DEresults){
  ego <- clusterProfiler::enrichGO(
    gene = rownames(comparison),
    universe = rownames(DEresults),
    OrgDb = org.Mm.eg.db,
    ont = "BP",
    keyType = "ENSEMBL",
    maxGSSize = 150,
    pvalueCutoff = 0.05
    )

  # filter for genes with log2FC higher than +/- 1 (2x DE)
  ego@result <- ego@result[which(-1 > ego@result$FoldEnrichment | 1 < ego@result$FoldEnrichment), ]

  return(ego)
}

ego_WT <- ego_gen(WT_Control_WT_Case, dss)
ego_DKO <- ego_gen(DKO_Control_DKO_Case, dss)
ego_Case <- ego_gen(WT_Case_DKO_Case, dss)
```

- *gene = sig*

  sig should be a vector of ENSEMBL gene IDs that are significantly DE.

- *universe = rownames(mat)*

  All genes measured in the experiment. Since mat is your DESeq2 input matrix, its row names are all ENSEMBL IDs → perfect universe.

- *OrgDb = org.Mm.eg.db*

  Genome-wide mouse annotation package from Bioconductor.

- *ont = "BP"* or "MF", "CC", or "ALL"

  GO subontology:

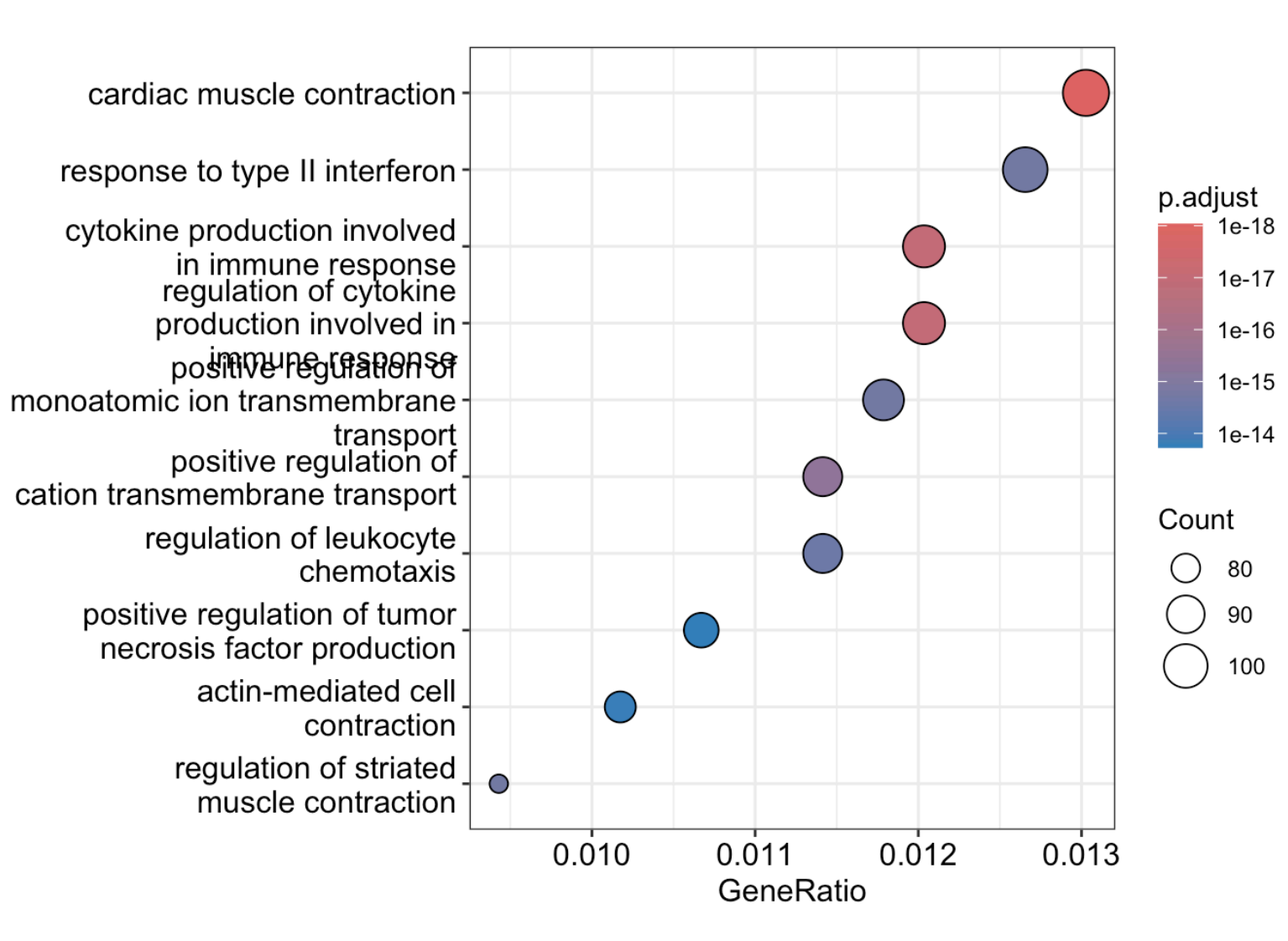
  - BP: Biological Process
  - MF: Molecular Function
  - CC: Cellular Component
- *keyType = "ENSEMBL"*

  Because your gene IDs (row names) are ENSEMBL IDs like "ENSMUSG00000000001".

- *maxGSSize = 150* maximal size of genes annotated for testing

Work on the results of enrichGO

```
dotplot(ego_WT)
```



cardiac muscle contraction
response to type II interferon
cytokine production involved in immune response
regulation of cytokine production involved in immune response
positive regulation of monoatomic ion transmembrane transport
positive regulation of cation transmembrane transport
regulation of leukocyte chemotaxis
positive regulation of tumor necrosis factor production
actin-mediated cell contraction
regulation of striated muscle contraction
0.010
0.011
0.012
0.013
GeneRatio
p.adjust
1e-18
1e-17
1e-16
1e-15
1e-14
Count
80
90
100